Tutorials (CE/CZ1006)
Tut 7

Prof Lin Weisi

Important concepts & methodology in this week

- *Cache memory (Q7.1)*
  - *Direct-mapped*
  - *N-way set associative cache*
  - *Fully associative cache*
    - *LRU*
    - *FIFO*
- *Virtual & physical memory (Q7.2)*
  - *Page table*
  - *Page fault*

Q7.1

System memory is the memory where the CPU execute the code and access the working copy of data during runtime.

Storage memory is where the computer stores the entire set of program and data. Majority of the time, storage memory do not support code execution.

In the example of a laptop, the system memory typically correspond to the DRAM connected to the processor.

The entire Microsoft Office Suite will be too large to be store in the system memory so is likely to be stored in the Storage memory, which is the HDD or SSD in the laptop.

During runtime, **part of the power point application** is loaded into the system for execution. **Any data (photos, graphics, video etc)** that is required to assemble the ppt slide would also need to be loaded into the system memory before the processor could perform any operation on it. Note that the **data need not be the entire photo/video**, only the subset that the processor works on need to be in the system memory at ay instance.

The final ppt file is stored in the storage memory.

Q7.2

SRAM. Cache needs to be implemented with **very fast** memory so as to keep up with the processor operating speed. DRAM is too slow and its transfer procedure too complex to be used for cache operation. Flash is not suitable due to its finite erase cycles and slower speed.

Q7.3

NAND:

- Data is accessed a page at a time. Command used to open a particular page followed by the individual bytes read/write. Does not support Execute in Place.
- More compact due to layout of the cells => more cost effective.
- Used mainly as storage memory. E.g. SSD, USB Flash Drive.

NOR:

- Allows Random Data Read by providing address information only.
- Supports Execute in Place. i.e. Code stored in NOR Flash can be executed directly without having to transfer to RAM first.
- Less compact than NAND flash.
- Used mainly as system memory to store program code.

Q7.4

a. HDD001:

Capacity = 4*1024*128*512 = 268, 435, 456 Bytes = 256Mbytes.
Note: 1Mbyte = 2^20 bytes

HDD002:

Capacity = 8*1024*256*512 = 1, 073, 741, 824 bytes = 1GByte.

Note:

1GByte = 2^30 bytes,

# of cylinders = # of tracks of each surface,

# of heads = # of valid surfaces

Q7.4

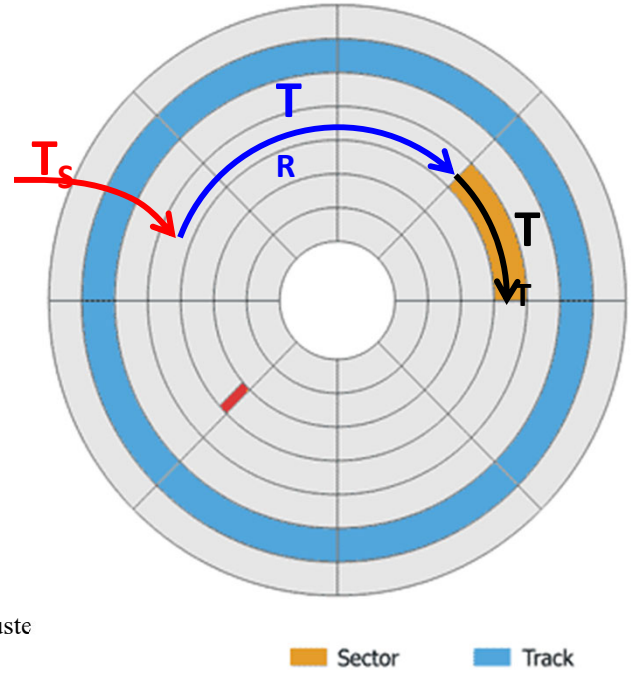
b. For HDD001

i. RPS = 5000/60

Average rotational delay = $T_{R,AV}$ = 0.5/RPS = 0.5/(5000/60) = 6ms

Seek time (track-to-track) = T_S = 5ms

Access time, $T_A = T_S + T_{R,AV}$ = 5 + 6 = 11ms



ii. RPS = 5000/60 per second
Access time T_A = 11 ms
# of sectors = 4*1024/512 = 8
Transfer time T_T for 1 sector = 512/(RPS*D_T*D_S) = 512/((5000/60)*128*512)
= 93.75 us
Total time, T_{TOTAL} = 8*(T_A + T_T) = 88.75 ms

Note: This illustrate the effect for HDD fragmentation. If a file is fragmented, i.e. stored in non-consecutive sectors/cluste
drop, as shown by the total access time for the two examples above.

iii. Defragmentation => file is stored in consecutive sectors.

However, 280KByte is large than size of one track (128*512=64KByte).

It occupies 4 full tracks and 24 Kbyte on the fifth track.

RPS = 5000/60 per second
Access time T_A = 11 ms. Note: T_A is required when moving from one track to the other since the R/W head need to be positioned at the starting sector of the next track.

T_T for one full track = 1/RPS = 12 ms

Total time for 1 track, T_{TOTAL} = 11+12 ms = 23 ms

4 tracks = 4*23 = 92ms

Last 24KByte:
Transfer time T_T = N/(RPS*DT*DS) = (24*1024)/((5000/60)*128*512) = 4.5 ms
Total time, T_{TOTAL} = 11+4.5ms = 15.5 ms

Total time needed to transfer 280KByte file = 92+15.5 = 107.5 ms

Q7.4

c. HDD002.

- Statistically more robust as MTTF = 1M hours, which is 2X that of HDD001. So less likely to fail.
- Higher storage
- Higher Performance (higher RPM, shorter Seek Time)

d.

- SSD still has a higher cost per bit than HDD. Not ideal for application that requires huge storage, especially for consumer home use.
- Finite read/write cycles of SSD compared to HDD's almost infinite read/write cycles.
- SSD cannot replace HDD in backup storage area, including data centers and servers, yet. This primarily due to cost.

Q7.5

System Memory: DRAM, Storage Memory: Magnetic HDD

- Entry level Desktop, General office use => low cost and only need average performance => DRAM and HDD as cost is lower than SSD
- Desktop => No much physical movement expected so robustness to movement is not required.
- Windows OS typically requires a few GByte of system memory. At this size, SRAM will be too expensive. Higher end product uses higher speed DRAM such as DDR3, DDR4 etc
- Large storage for videos => Magnetic HDD (lower cost per bit than SSD)

Q7.6

- Redundancy.
  - User data is stored in multiple servers at multiple sites. If one breaks down, the other would take over.
  - Within a server, some other error correction/recovery techniques may be used to mitigate localized error.
  - Data is also backup to tapes. This serves as the last line of defense.

Q7.7

Flash has a larger page size (order of Kbytes) so can only be erased at Kbytes level. This is not suitable for certain use case such as storing of user configuration parameters, which is typically done at order of bytes level. An EEPROM, which has a smaller page size (tens of bytes) is more suitable for such use case as it result in less page erases. Also, EEPROM has a larger erase cycle endurance i.e. it could tolerate larger number of erasures before failing.